Lab - 3: **Experiment No: 3** **Dated:**

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Lab Objectives: To implement public and private Blockchain.

***Lab Outcomes (LO)*:** Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Download the code from folder, **Lab_3**
2. Install requests in the virtual environment created in the Lab 2. (Follow the instructions)
3. Run the files - **hadcoin_node_5001.py**, **hadcoin_node_5002.py**, **hadcoin_node_5003.py** in 3 different terminals.
4. Open Postman, from each node - invoke **connect_node()** and pass the peers as POST requests.
5. Perform the following functions
   - Add Transactions - invoke add_transactions() as a POST request.
   - mining - mine_block(),
   - fetch the chain - get_chain(),
   - replace the longest chain - replace_chain()
6. Modify the code such that transactions are removed after they are added to the block.

Tools & Libraries used :

- Install ***Flask*** : pip install Flask
- Download ***Postman*** from https://www.postman.com/
- Python Libraries : **datetime, jsonify, hashlib, uuid4, urlparse, request**
- **Install requests :** pip install requests==2.18.4

Instructions : (Prepare for viva for the following topics)

1. Challenges in P2P networks
2. How transactions are performed on the network?
3. Explain the role of mempools
4. Write briefly about the libraries and the tools used during implementation.

Outcome :

1. Understood the challenges in P2P networks, how transactions are performed and how a miner mines a block to be added in a blockchain.
2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.
3. Successfully mined the blocks among a P2P network with 3 nodes.
4. Performed transactions via the network.
5. Successfully updated the block across the network
6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.

7. Submit the hardcopy **by Sem** (As per the instructions, submit a hard copy of the same).

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Guidelines

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Download the code from folder, Lab_3
2. Install requests in the virtual environment created in the Lab 2. (Follow the instructions)
3. Run the files - hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py in 3
different terminals.
4. Open Postman, from each node - invoke connect_node() and pass the peers as POST requests.
5. Perform the following functions
    - Add Transactions - invoke add_transactions() as a POST request.
    - mining - mine_block(),
    - fetch the chain - get_chain(),
    - replace the longest chain - replace_chain()
6. Modify the code such that transactions are removed after they are added to the block.

Tools & Libraries used :

- Install Flask : pip install Flask
- Download Postman from https://www.postman.com/
- Python Libraries : datetime, jsonify, hashlib, uuid4, urlparse, request
- Install requests : pip install requests==2.18.4

Instructions : (Prepare for viva for the following topics)

1. Challenges in P2P networks
2. How transactions are performed on the network?
3. Explain the role of mempools
4. Write briefly about the libraries and the tools used during implementation.

Outcome :

1. Understood the challenges in P2P networks, how transactions are performed and how a

miner mines a block to be added in a blockchain.

2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.

3. Successfully mined the blocks among a P2P network with 3 nodes.

4. Performed transactions via the network.

5. Successfully updated the block across the network

6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.

7. Submit the hardcopy by the 2nd week of August 2023

(As per the instructions, submit a hard copy of the same).

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.
Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (`5001`, `5002`, `5003`).
Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
  `pip install Flask`
- **Requests** – For HTTP communication between nodes
  `pip install requests==2.18.4`
- **Postman** – For testing API requests
- Python Standard Libraries:
  - `datetime`
  - `jsonify`
  - `hashlib`
  - `uuid4`
  - `urlparse`
  - `request`

Procedure and Output:

1. Download `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`.
2. Install required packages.
3. Run each node in separate terminals.
4. Connect nodes using Postman – POST `/connect_node`.

{

"nodes":

[

"http://127.0.0.1:5002",

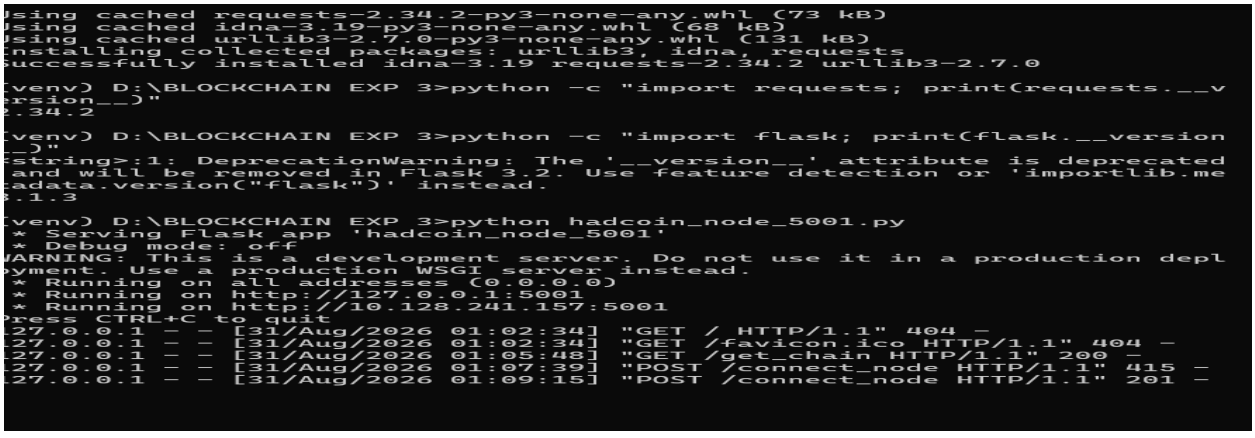
"http://127.0.0.1:5002"] }

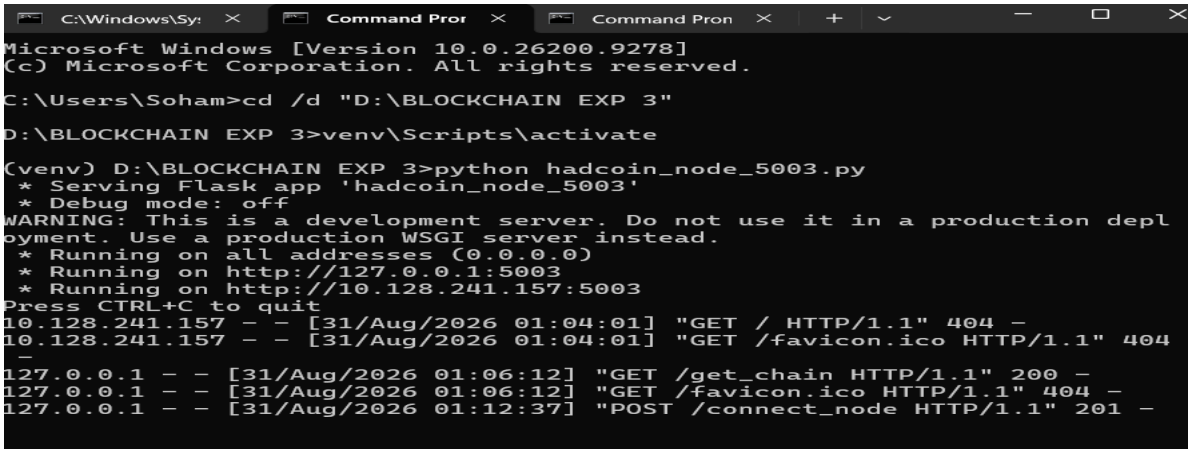
5. Add transactions – POST
6. Add 3 transactions in 5001
7. Condition 2 — Each node must have **different chain lengths**

Example:

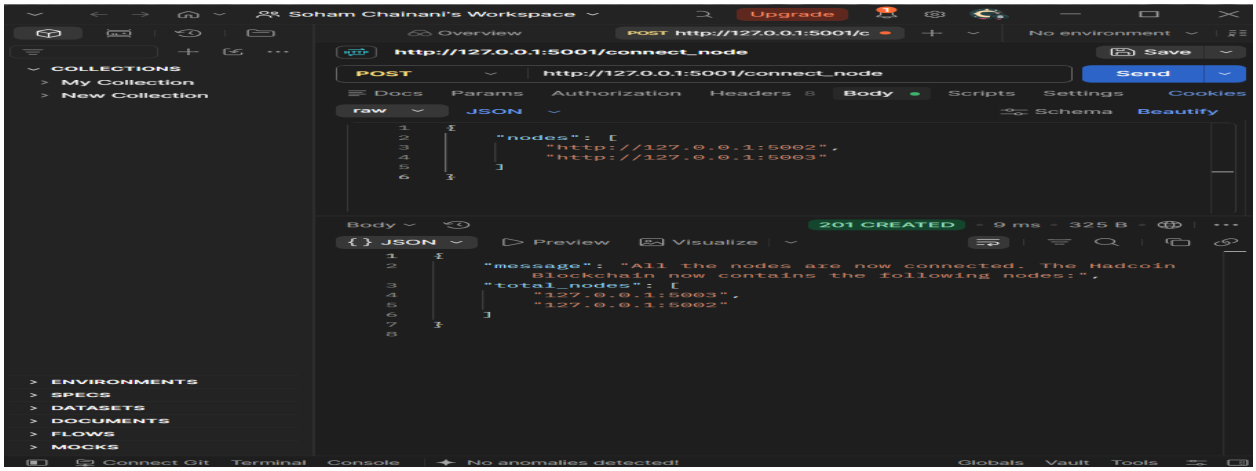
If you mined 1 block on 5001, but did not mine on 5002 or 5003 →

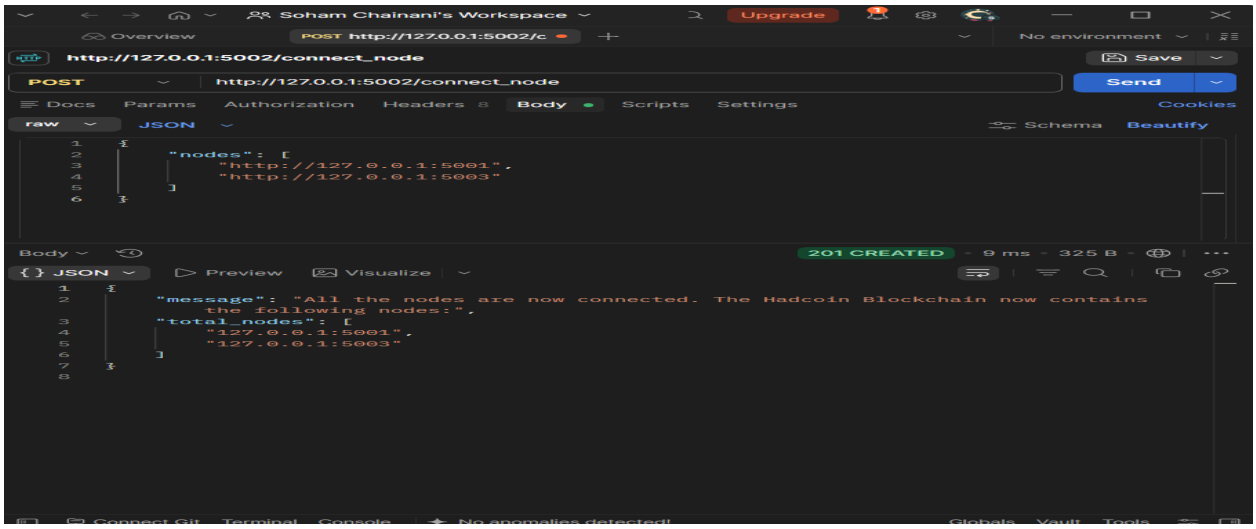
- **5001 chain length = 2**
- **5002 chain length = 1**
- **5003 chain length = 1**

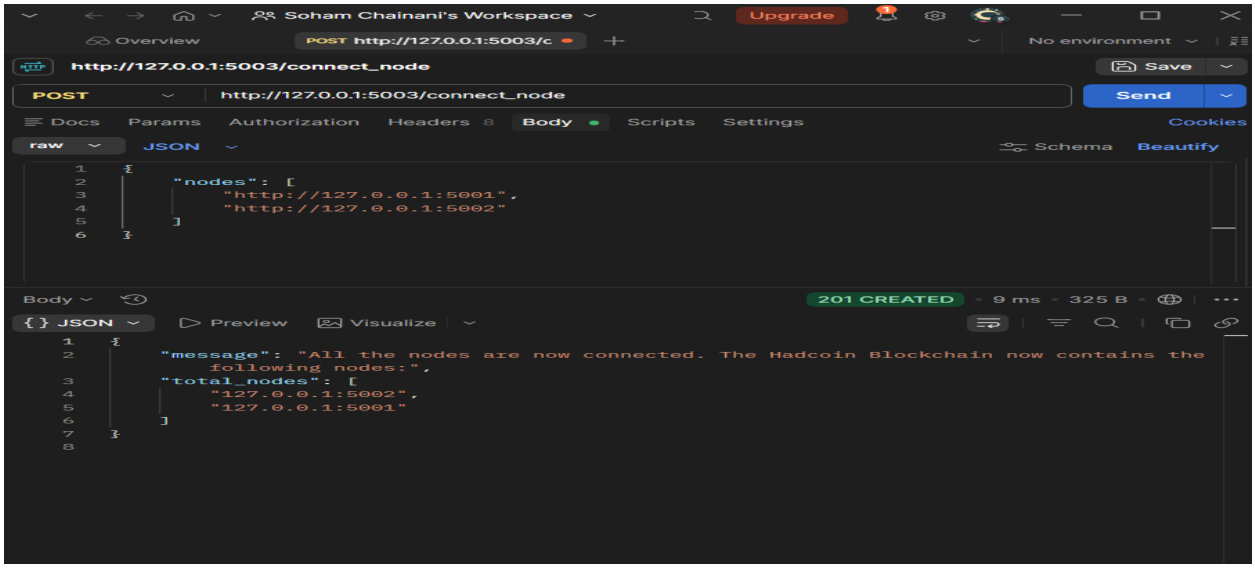




```
C:\Windows\Sy:   Command Pron   Command Pror

Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Soham>cd /d "D:\BLOCKCHAIN EXP 3"

D:\BLOCKCHAIN EXP 3>venv\Scripts\activate

(venv) D:\BLOCKCHAIN EXP 3>python hadcoin_node_5002.py
 * Serving Flask app 'hadcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production depl
oyment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://10.128.241.157:5002
Press CTRL+C to quit
127.0.0.1 - - [31/Aug/2026 01:04:41] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [31/Aug/2026 01:06:02] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [31/Aug/2026 01:11:40] "POST /connect_node HTTP/1.1" 201 -
```







STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → `http://127.0.0.1:5001/add_transaction`

```
{
  "sender": "a",
  "receiver": "b",
  "amount": 5
}
```

Then 5001 → GET `/mine_block`

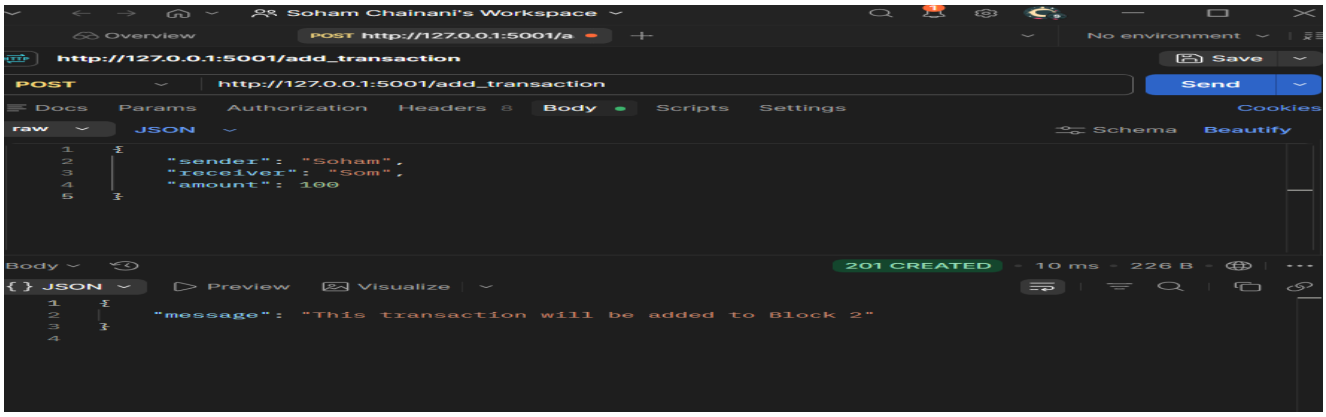
STEP 3 — Now go to 5002 and run:

GET → `/replace_chain`

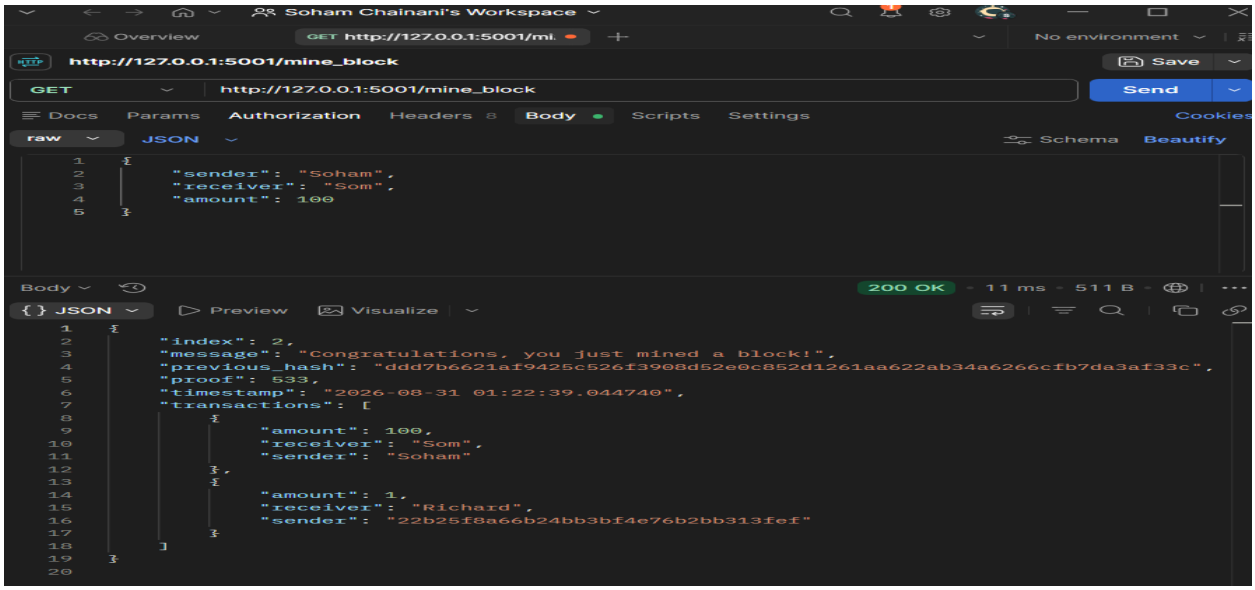
STEP 4 — Repeat on 5003

GET → `/replace_chain`

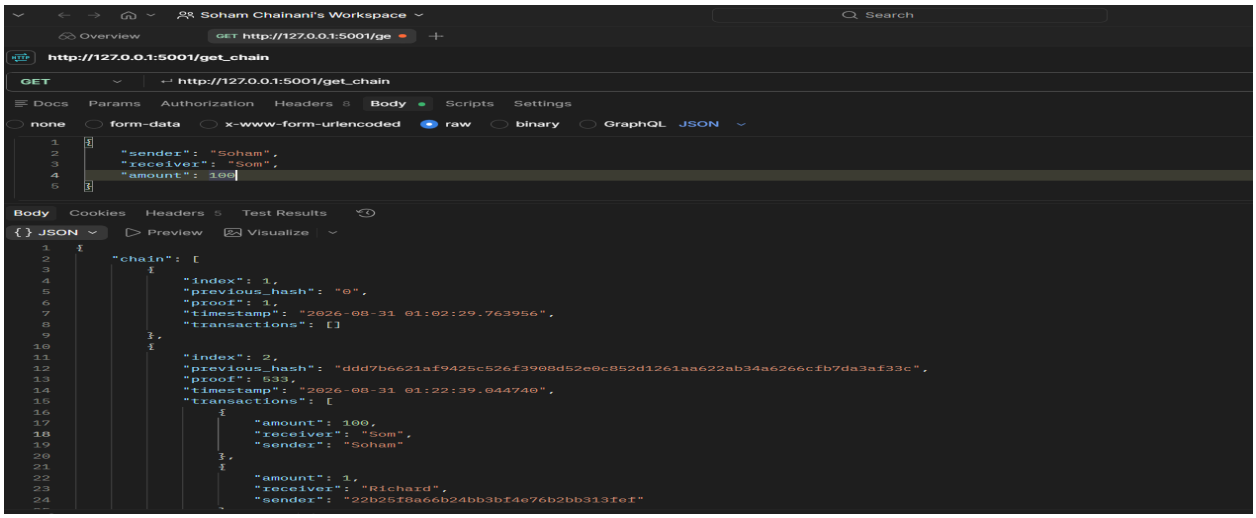
Same result.

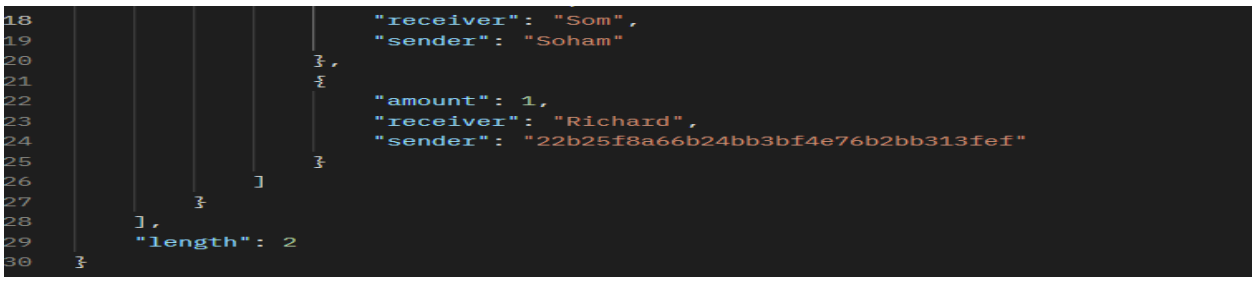


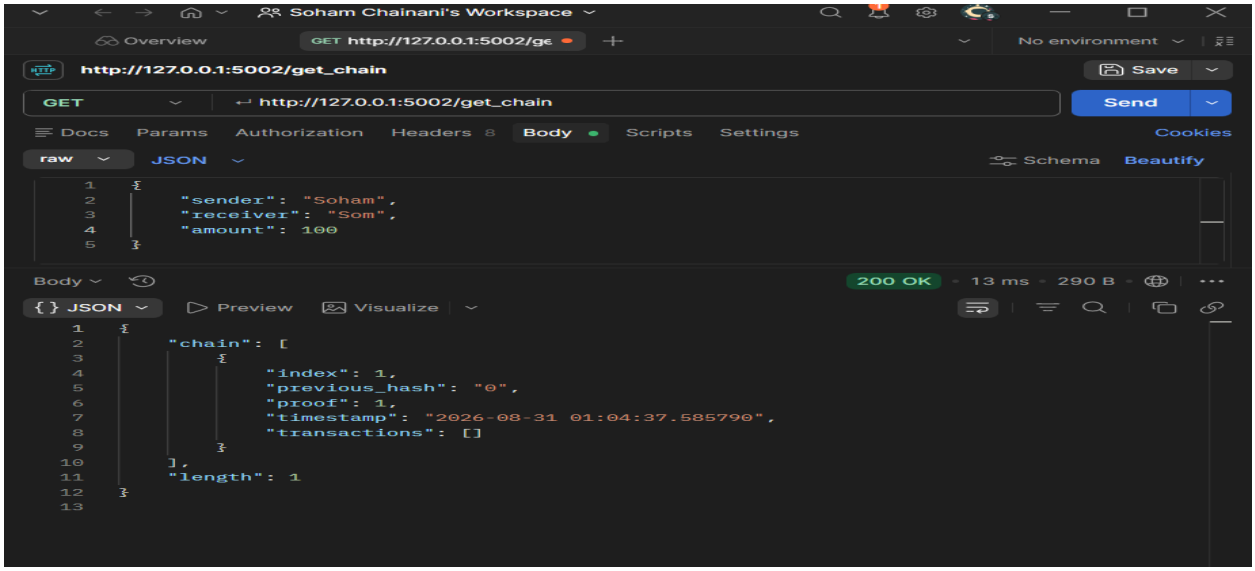
8. Mine blocks – GET `/mine_block`.

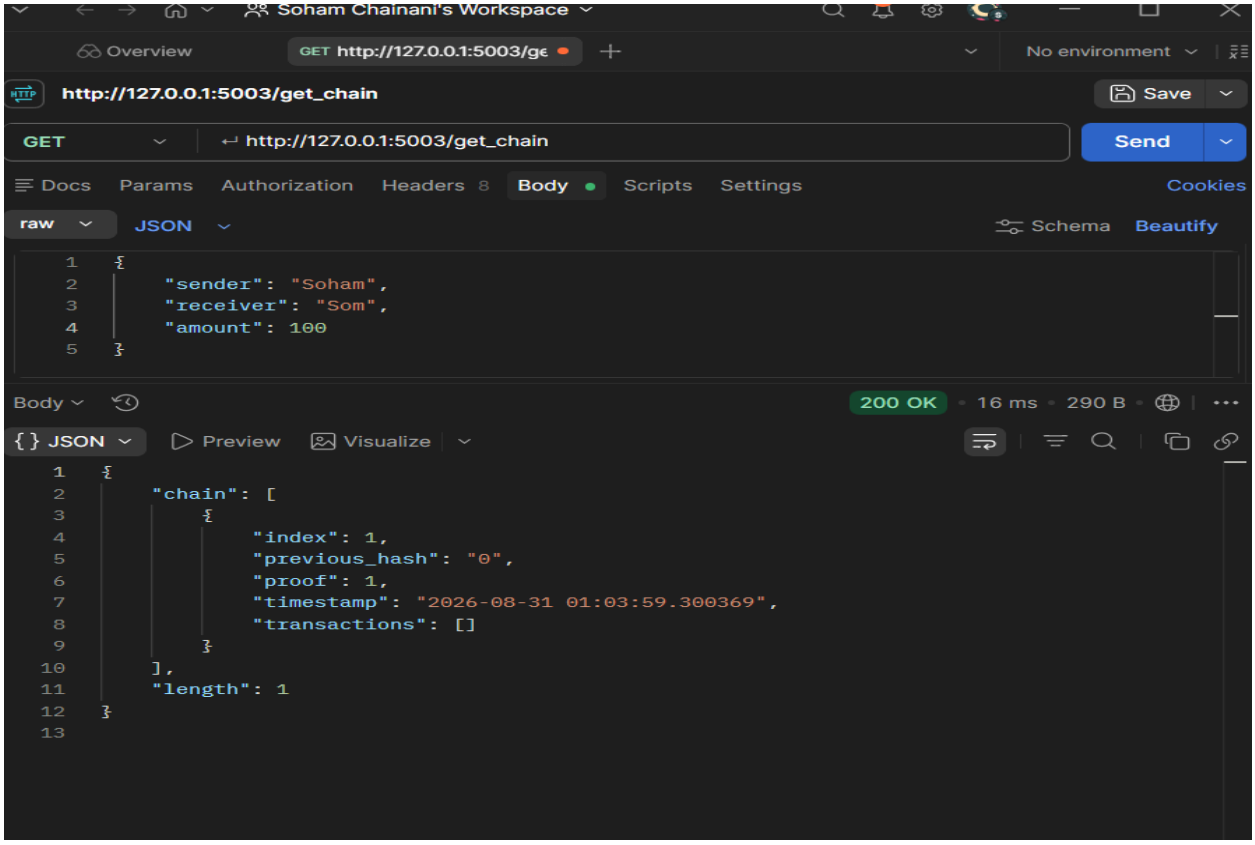


9. Fetch blockchain – GET `/get_chain`.

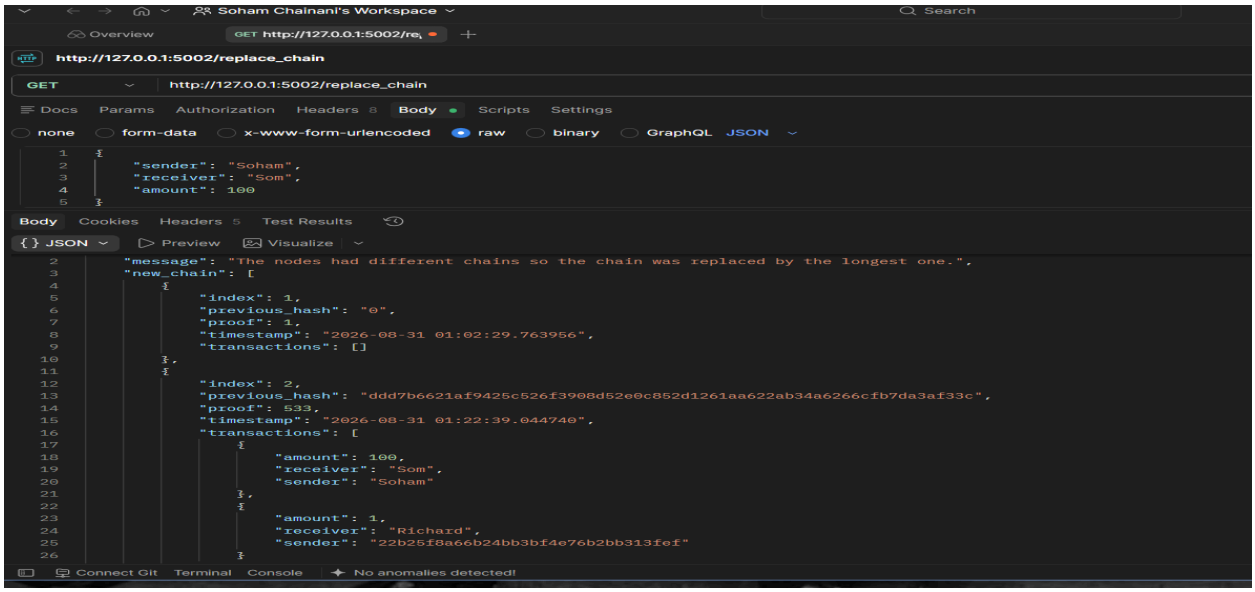

Soham Chainani's Workspace
Search
Overview
GET http://127.0.0.1:5001/ge
http://127.0.0.1:5001/get_chain
GET
http://127.0.0.1:5001/get_chain
Docs Params Authorization Headers 8 Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
{
"sender": "Soham",
"receiver": "Som",
"amount": 100
}
Body Cookies Headers 5 Test Results
{} JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:02:29.763956",
"transactions": []
},
{
"index": 2,
"previous_hash": "ddd7b6621af9425c526f3908d52e0c852d1261aa622ab34a6266cfb7da3af33c",
"proof": 533,
"timestamp": "2026-08-31 01:22:39.044740",
"transactions": [
{
"amount": 100,
"receiver": "Som",
"sender": "Soham"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "22b25f8a66b24bb3bf4e76b2bb313fef"

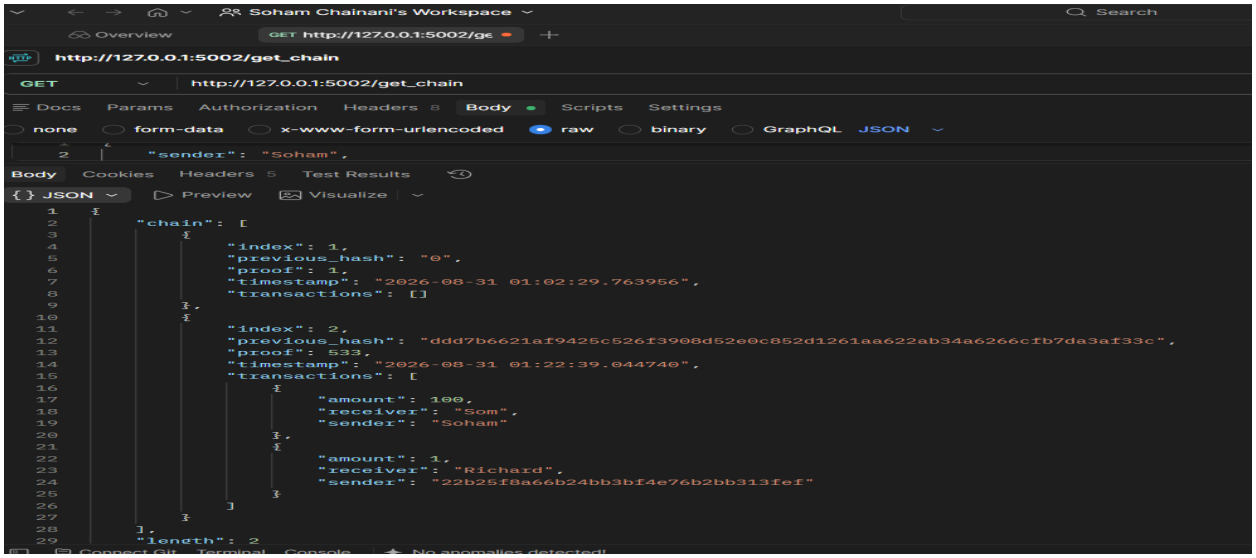

"receiver": "Som",
"sender": "Soham"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "22b25f8a66b24bb3bf4e76b2bb313fef"
}
]
}
],
"length": 2
}

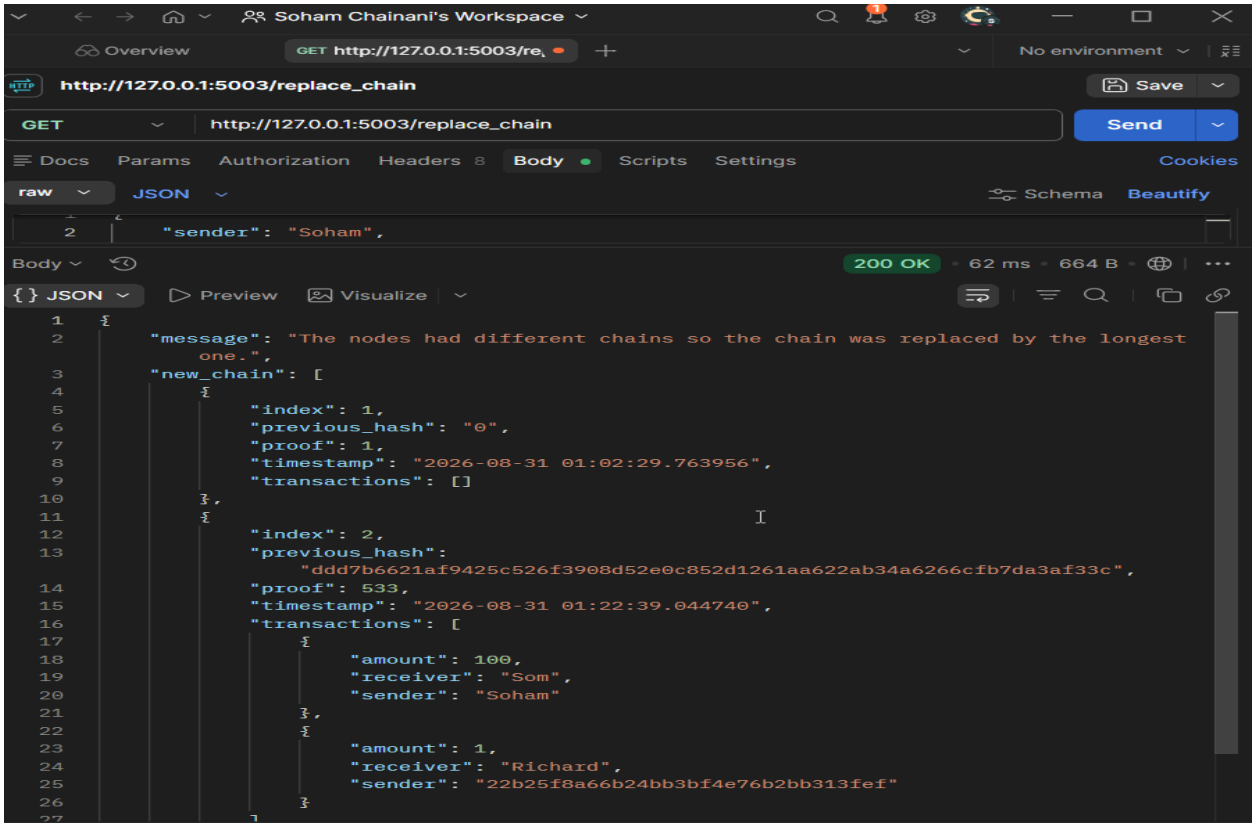

Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5002/ge
No environment
http://127.0.0.1:5002/get_chain
Save
GET
http://127.0.0.1:5002/get_chain
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
raw JSON
Schema Beautify
{
"sender": "Soham",
"receiver": "Som",
"amount": 100
}
Body
200 OK 13 ms 290 B
{} JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:04:37.585790",
"transactions": []
}
],
"length": 1
}

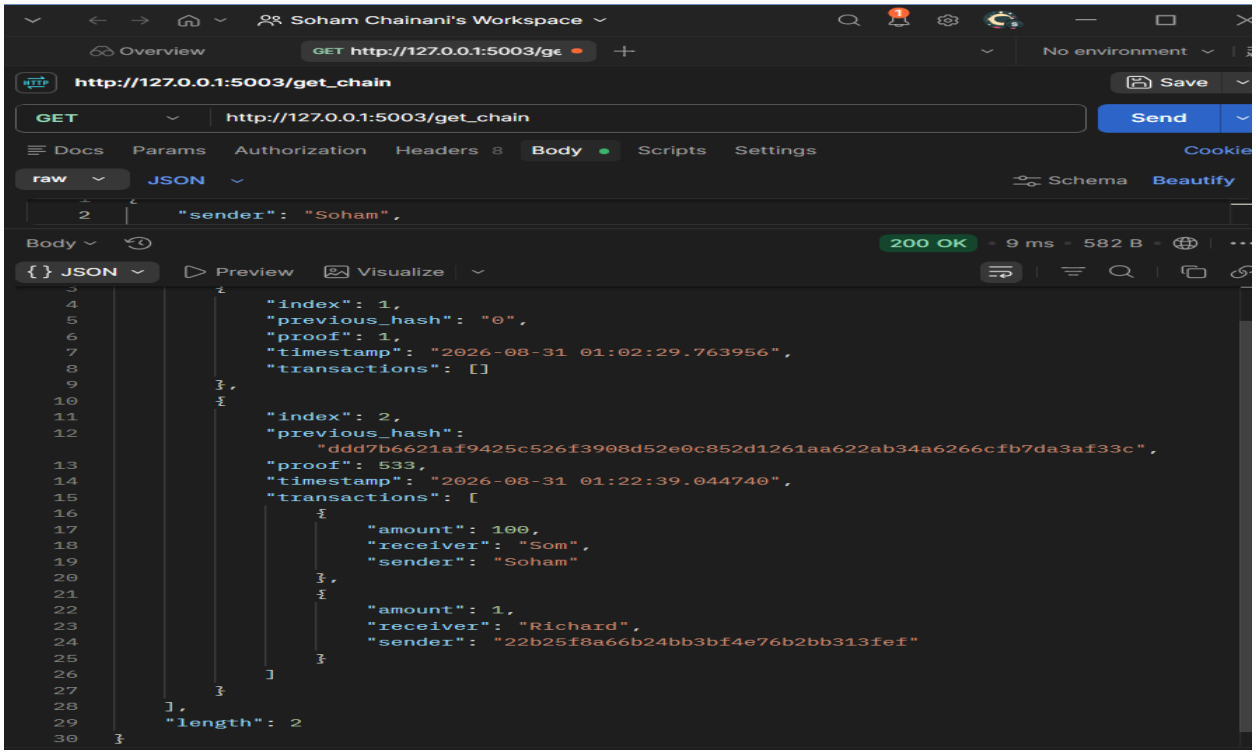


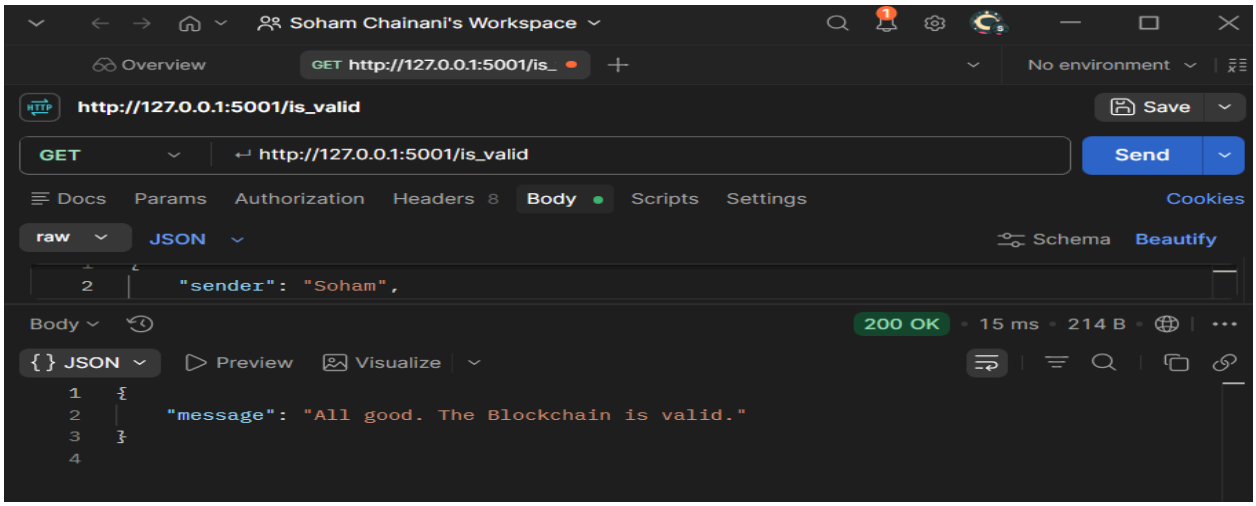
10. Replace chain – GET `/replace_chain`

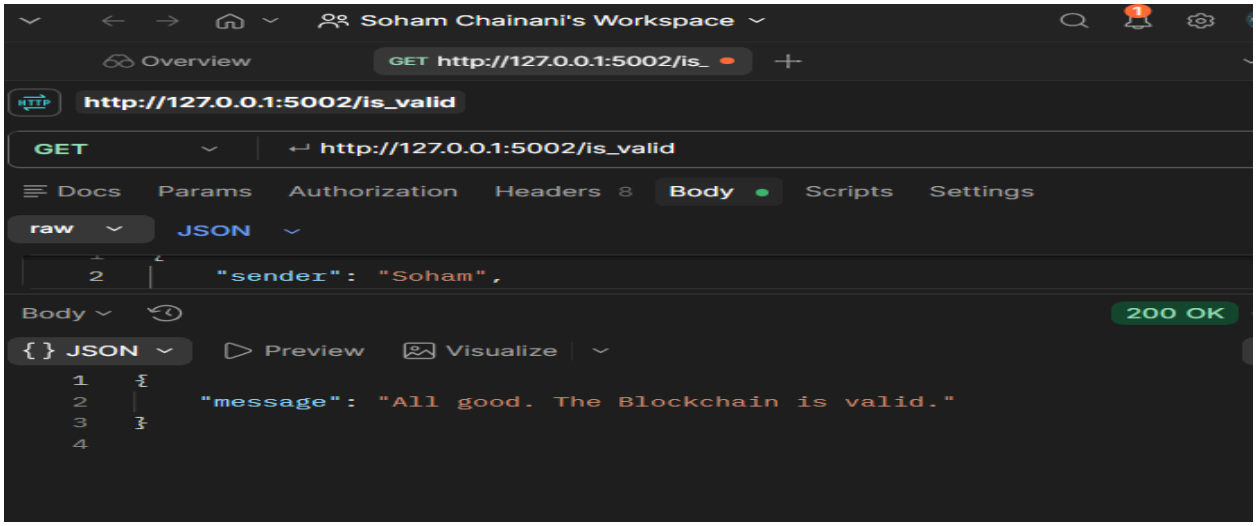


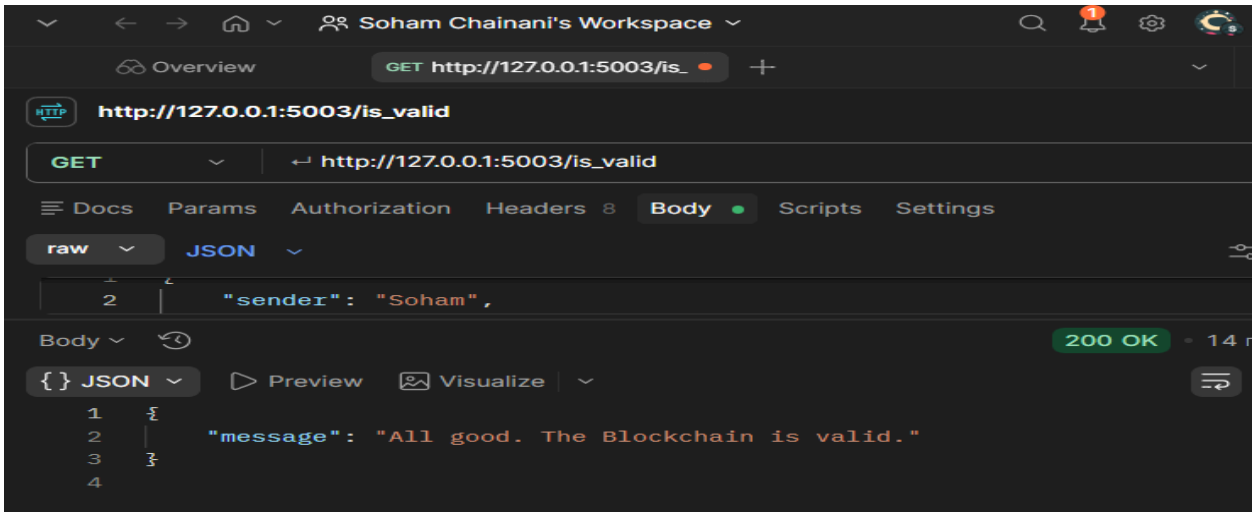

Soham Chainani's Workspace
Search
Overview
GET http://127.0.0.1:5002/ge
http://127.0.0.1:5002/get_chain
GET
http://127.0.0.1:5002/get_chain
Docs Params Authorization Headers 8 Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
2 "sender": "Soham",
Body Cookies Headers 5 Test Results
{ } JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:02:29.763956",
"transactions": []
},
{
"index": 2,
"previous_hash": "ddd7b6621af9425c526f3908d52e0c852d1261aa622ab34a6266cfb7da3af33c",
"proof": 533,
"timestamp": "2026-08-31 01:22:39.044740",
"transactions": [
{
"amount": 100,
"receiver": "Som",
"sender": "Soham"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "22b25f8a66b24bb3bf4e76b2bb313fef"
}
]
}
],
"length": 2

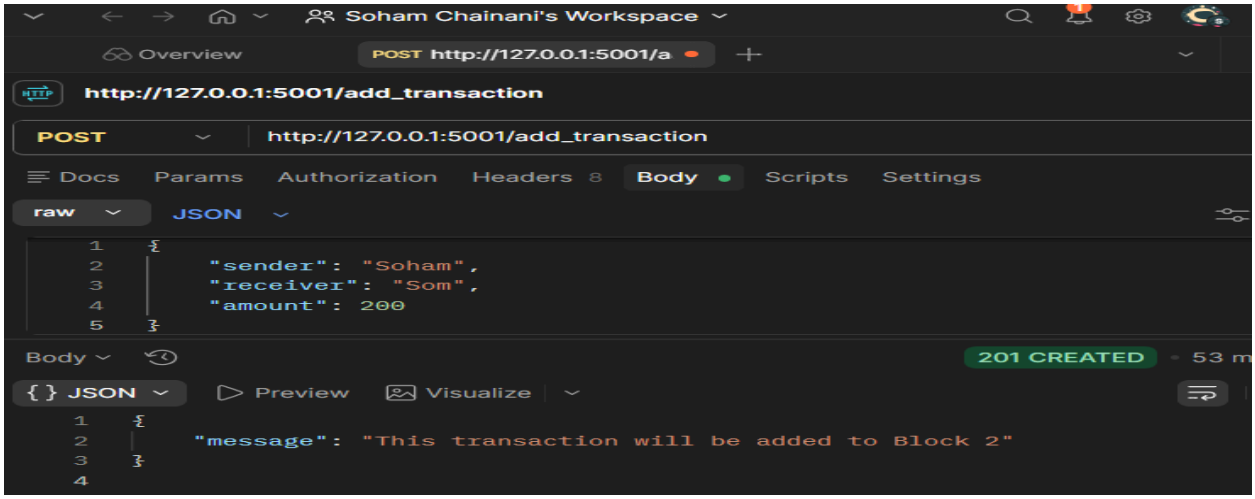

Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5003/re
No environment
http://127.0.0.1:5003/replace_chain
Save
GET
http://127.0.0.1:5003/replace_chain
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
raw JSON
Schema Beautify
2 "sender": "Soham",
Body
200 OK 62 ms 664 B
{ } JSON Preview Visualize
{
"message": "The nodes had different chains so the chain was replaced by the longest one.",
"new_chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:02:29.763956",
"transactions": []
},
{
"index": 2,
"previous_hash": "ddd7b6621af9425c526f3908d52e0c852d1261aa622ab34a6266cfb7da3af33c",
"proof": 533,
"timestamp": "2026-08-31 01:22:39.044740",
"transactions": [
{
"amount": 100,
"receiver": "Som",
"sender": "Soham"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "22b25f8a66b24bb3bf4e76b2bb313fef"
}

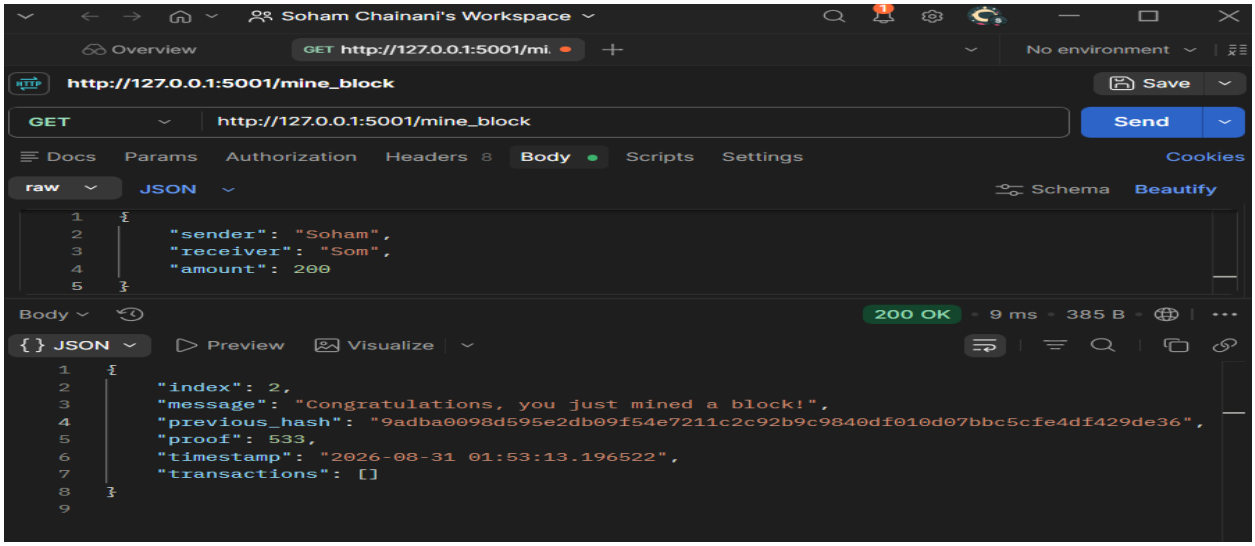

Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5003/ge
No environment
http://127.0.0.1:5003/get_chain
Save
GET
http://127.0.0.1:5003/get_chain
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookie
raw JSON
Schema Beautify
2 "sender": "Soham",
Body
200 OK 9 ms 582 B
{ } JSON Preview Visualize
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:02:29.763956",
"transactions": []
"index": 2,
"previous_hash":
"ddd7b6621af9425c526f3908d52e0c852d1261aa622ab34a6266cfb7da3af33c",
"proof": 533,
"timestamp": "2026-08-31 01:22:39.044740",
"transactions": [
"amount": 100,
"receiver": "Som",
"sender": "Soham"
"amount": 1,
"receiver": "Richard",
"sender": "22b25f8a66b24bb3bf4e76b2bb313fef"
],
"length": 2

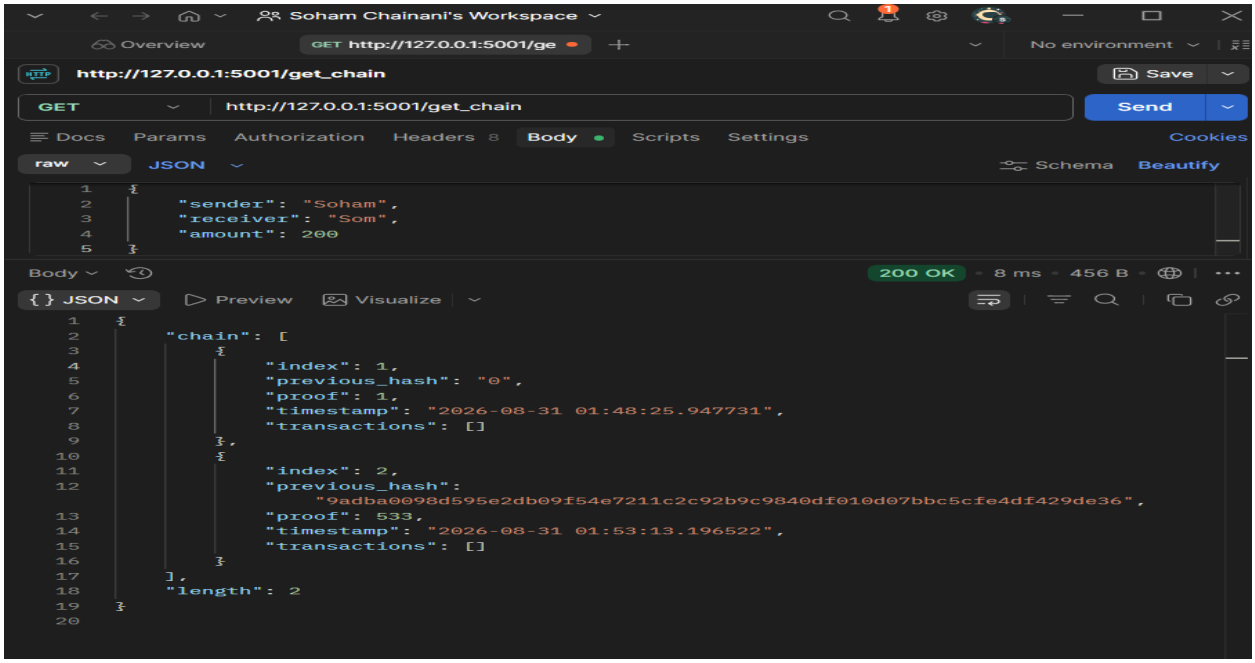

Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5001/is_
No environment
http://127.0.0.1:5001/is_valid
Save
GET
http://127.0.0.1:5001/is_valid
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
raw JSON
Schema Beautify
2 "sender": "Soham",
Body
200 OK 15 ms 214 B
{ } JSON Preview Visualize
{
"message": "All good. The Blockchain is valid."
}

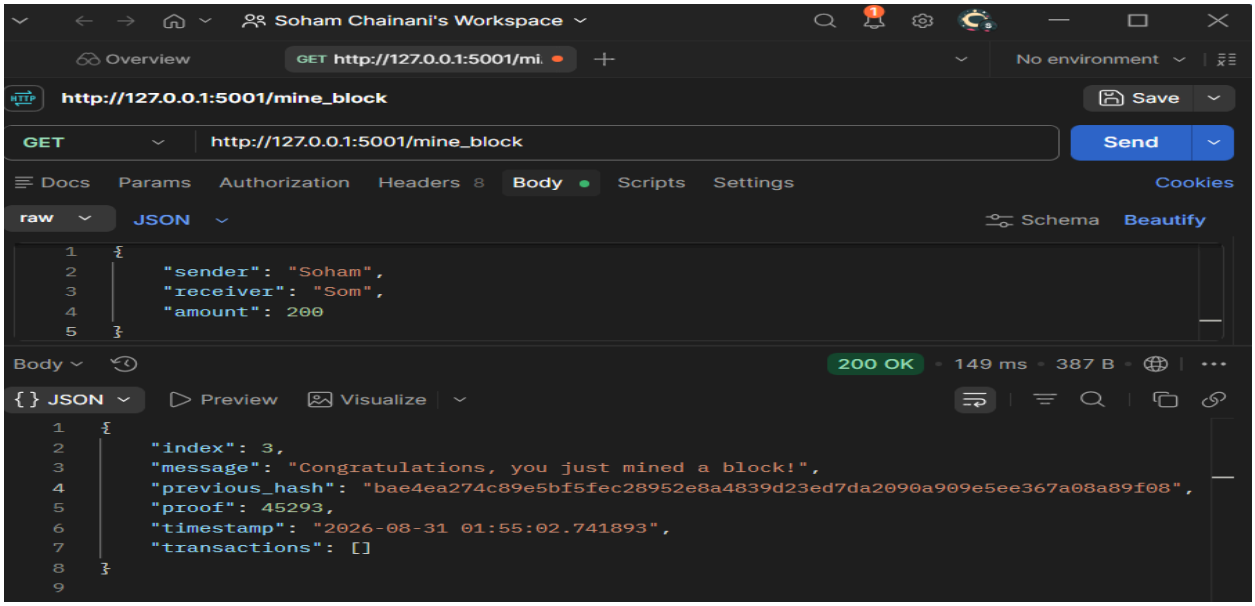

Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5002/is_
http://127.0.0.1:5002/is_valid
GET
http://127.0.0.1:5002/is_valid
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
raw
JSON
2 "sender": "Soham",
Body
200 OK
{ } JSON
Preview
Visualize
1 {
2 "message": "All good. The Blockchain is valid."
3 }
4


Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5003/is_
http://127.0.0.1:5003/is_valid
GET
http://127.0.0.1:5003/is_valid
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
raw
JSON
2 "sender": "Soham",
Body
200 OK
14
{ } JSON
Preview
Visualize
1 {
2 "message": "All good. The Blockchain is valid."
3 }
4


Soham Chainani's Workspace
Overview
POST http://127.0.0.1:5001/a
http://127.0.0.1:5001/add_transaction
POST
http://127.0.0.1:5001/add_transaction
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
raw
JSON
1 {
2 "sender": "Soham",
3 "receiver": "Som",
4 "amount": 200
5 }
Body
201 CREATED
53 m
{ } JSON
Preview
Visualize
1 {
2 "message": "This transaction will be added to Block 2"
3 }
4


Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5001/mi
No environment
http://127.0.0.1:5001/mine_block
Save
GET
http://127.0.0.1:5001/mine_block
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"sender": "Soham",
"receiver": "Som",
"amount": 200
}
Body
200 OK 9 ms 385 B
{ } JSON
Preview
Visualize
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "9adba0098d595e2db09f54e7211c2c92b9c9840df010d07bbc5cfe4df429de36",
"proof": 533,
"timestamp": "2026-08-31 01:53:13.196522",
"transactions": []
}


Soham Chainani's Workspace
Overview
GET http://127.0.0.1:5001/ge
No environment
http://127.0.0.1:5001/get_chain
Save
GET
http://127.0.0.1:5001/get_chain
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"sender": "Soham",
"receiver": "Som",
"amount": 200
}
Body
200 OK 8 ms 456 B
{ } JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 01:48:25.947731",
"transactions": []
},
{
"index": 2,
"previous_hash":
"9adba0098d595e2db09f54e7211c2c92b9c9840df010d07bbc5cfe4df429de36",
"proof": 533,
"timestamp": "2026-08-31 01:53:13.196522",
"transactions": []
}
],
"length": 2
}



Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.